encamina
PIENSA EN COLORES

Diseño de contratos OpenAPI con Spec Kit

Manual del taller greenfield | Open Finance

Metodología	Spec Kit (Greenfield)
Entorno	API-Led / GitLab / Jenkins
Validación	Stoplight Spectral y Redocly CLI
Entregable	Contrato OpenAPI consolidado, sin codigó



ENCAMINA

Octubre 2026

Valencia (sede central)
Av. Jerónimo Roure, 49
46520, Sagunto (Valencia)
Telf. +34 962 698 064

Madrid
C/ O'Donnell, 34
28009, Madrid
Telf. +34 917 893 823

Murcia
C/ del Ángel, 32
30202 Cartagena (Murcia)

Tenerife
Av. Trinidad, 61. Campus Central ULL
38200, S. Cristóbal de La Laguna (Tenerife)

info@encamina.com
www.encamina.com

Diseño de contratos OpenAPI con Spec Kit

Índice

1 Objetivo 3
2 Participantes y preparación 3
3 Alcance y entregables 3
4 Las 7 C de la comunicación 4
5 Arquitectura API-Led y nomenclatura 4
5.1 Responsabilidades 4
5.2 Convención de nombres de Open Finance 4
6 Caso práctico: resumen de inversiones 5
7 Organización en GitLab 6
7.1 Estructura del proyecto contractual 6
7.2 Componentes compartidos 7
8 Desarrollo del taller con Spec Kit 8
8.1 Organización de Spec Kit por API 8
8.2 Cuántas constituciones se deben definir 9
8.3 Cómo especializar los capítulos 9
8.4 Guía para las instrucciones personalizadas de Copilot 10
8.5 Paso 1. Preparar el gobierno y generar la constitución efectiva 11
8.6 Actualización de las reglas compartidas 12
8.7 Paso 2. Redactar la especificación 12
8.8 Paso 3. Resolver las ambigüedades 13
8.9 Paso 4. Preparar GitLab y planificar el diseño contractual 13
8.10 Paso 5. Agente Implementar contrato y skills necesarias 14
8.11 Paso 6. Validar y consolidar 17
8.12 Paso 7. Revisar y aprobar 17
9 Validación y consolidación con Jenkins 17
9.1 Uso del contrato en la pipeline de publicación 18
10 Criterios de aceptación 18
11 Resultado esperado 19

1 Objetivo

Transformar una especificación funcional de una API de Open Finance en un contrato OpenAPI modular, completo y verificable mediante un enfoque greenfield. La especificación de entrada deberá cumplir las 7 C de la comunicación y permitir generar el contrato sin introducir requisitos no acordados.

El resultado del taller es exclusivamente el diseño contractual. No se generarán endpoints ejecutables, servicios, controladores, acceso a datos ni código de implementación de la API.

El contexto de trabajo contempla una arquitectura API-Led, repositorios de contratos en GitLab y automatización mediante Jenkins, Stoplight Spectral y Redocly CLI.

2 Participantes y preparación

El taller está dirigido a analistas funcionales, arquitectos y diseñadores de APIs. Los participantes deberán disponer de:

- Un repositorio de trabajo y acceso a GitHub Copilot con los flujos de Spec Kit disponibles.
- Las convenciones de diseño y el catálogo de dominios acordados para el ejercicio.
- Acceso a las revisiones aprobadas de los componentes compartidos utilizados.
- Un entorno local o Jenkins con versiones fijadas y compatibles de Spectral y Redocly CLI.

No es necesario disponer de un backend para completar el ejercicio. La duración se ajustará al número de operaciones y al detalle del caso seleccionado.

3 Alcance y entregables

Entregable	Contenido
Especificación funcional	Propósito, consumidores, operaciones, datos, reglas, errores y criterios de aceptación.
Decisiones de diseño	Capa API-Led, nomenclatura, organización contractual y dependencias compartidas.
Contrato modular	Documento OpenAPI raíz y archivos de recursos, parámetros y esquemas referenciados.
Evidencias de validación	Informes de Spectral y Redocly y trazabilidad entre requisitos y operaciones.
Contrato consolidado	Un único documento OpenAPI generado y validado, preparado como entrada de la pipeline de publicación en el gestor de APIs seleccionado.

Quedan fuera del alcance la implementación de la API, las bases de datos, la generación de SDK y el despliegue de servicios. El taller preparará el contrato consolidado y validado para su consumo por una pipeline posterior de publicación en un gestor de APIs, como Amazon API Gateway, WSO2 API Manager o Azure API Management. La ejecución de esa publicación requerirá un flujo independiente y aprobado.

4 Las 7 C de la comunicación

Las 7 C se utilizarán para revisar la comunicación de requisitos, no como sustituto de la validación técnica de OpenAPI.

Principio	Aplicación a la especificación	Comprobación
Claridad	Expresar el propósito y comportamiento sin ambigüedades.	Cada operación tiene una responsabilidad identificable.
Concisión	Eliminar repeticiones e información irrelevante.	Cada requisito aporta una condición necesaria.
Concreción	Precisar rutas, métodos, tipos, formatos y restricciones.	Se pueden derivar ejemplos y comprobaciones objetivas.
Corrección	Utilizar términos de negocio y reglas técnicas válidos.	Datos y respuestas respetan los estándares acordados.
Coherencia	Mantener las mismas convenciones y significados.	Un concepto conserva su nombre, tipo y significado.
Completitud	Incluir entradas, salidas, errores, seguridad y casos límite relevantes.	No faltan decisiones necesarias para generar el contrato.
Cortesía	Redactar de forma profesional y orientada al consumidor.	Las descripciones son respetuosas, comprensibles y útiles.

REGLA DEL 95 % · ACLARACIONES Y CONFIANZA MÍNIMA

Antes de generar artefactos se aplicará la regla de aclaraciones del repositorio: alcanzar al menos un 95 % de confianza sobre lo solicitado. Si falta una decisión relevante, se formulará una pregunta concreta y se incorporará la respuesta a la especificación. No se inventarán reglas de negocio, direcciones de servidores ni modelos corporativos.

5 Arquitectura API-Led y nomenclatura

5.1 Responsabilidades

- **Experience API (`exp`)**: define un contrato adaptado a un consumidor o experiencia concretos.
- **Process API (`prc`)**: define una capacidad que requiere orquestación o coordinación de negocio.
- **System API (`sys`)**: define el acceso contractual a capacidades de un sistema subyacente.

Cada capacidad expuesta al consumidor tendrá un contrato de experiencia definido. Se identificarán tantas System APIs como sean necesarias y se añadirá una Process API únicamente cuando exista una responsabilidad de orquestación justificada.

No se crearán las tres capas automáticamente. Una Experience API puede consumir directamente una System API. Los contratos Process y System pueden reutilizarse entre diferentes experiencias.

5.2 Convención de nombres de Open Finance

REGLA ESTRICTA DE NOMENCLATURA

Se utilizará exclusivamente la convención `of-<capa>-<dominio>-<capacidad>-v<versión>` para los ejemplos del taller.

Elemento	Regla	Ejemplo
Prefijo	Identificar Open Finance.	`of`
Capa	Utilizar `exp`, `prc` o `sys`.	`prc`
Dominio	Utilizar un código del catálogo acordado.	`port`
Capacidad	Expresar la responsabilidad en inglés y con guiones.	`investment-summary`
Versión	Indicar la versión principal del contrato.	`v1`

Los nombres estarán en inglés y minúsculas, sin tecnologías ni entornos. La versión completa del contrato se declarará en `info.version`; el sufijo del identificador reflejará su versión principal. Cualquier versión en rutas se acordará explícitamente.

Para el ejercicio se utilizarán `port` para inversiones y carteras y `acct` para cuentas. Antes de adoptar estos códigos fuera del taller, deberán contrastarse con el catálogo corporativo aprobado.

Capa	Identificador	Finalidad
Experience	`of-exp-port-investment-dashboard-v1`	Presentar el resumen de inversiones a un canal digital.
Process	`of-prc-port-investment-summary-v1`	Coordinar la composición del resumen de inversiones.
System	`of-sys-port-portfolio-v1`	Exponer capacidades del sistema de carteras.
System	`of-sys-acct-core-banking-v1`	Exponer capacidades del sistema de cuentas.

El proyecto GitLab, el documento OpenAPI raíz y el artefacto consolidado compartirán el identificador de la API correspondiente.

6 Caso práctico: resumen de inversiones

El ejercicio consiste en especificar y generar el contrato de `of-prc-port-investment-summary-v1`, una Process API que expone un resumen de inversiones a partir de capacidades de carteras y cuentas.

El contrato de experiencia `of-exp-port-investment-dashboard-v1` consume la capacidad de resumen. La Process API coordina las capacidades de `of-sys-port-portfolio-v1` y `of-sys-acct-core-banking-v1`.

Estas relaciones describen el contexto arquitectónico, no llamadas implementadas. El ejercicio inicial puede limitarse al contrato de la Process API. Si la especificación aprobada requiere diseñar también la Experience API y las System APIs, se generará un contrato independiente por API siguiendo el procedimiento del taller. No se crearán APIs ni capas por el mero hecho de aparecer en el esquema.

Antes de diseñar el contrato, el equipo deberá acordar:

1. El consumidor y el propósito del resumen de inversiones.
2. Las operaciones expuestas, con sus métodos y rutas exactos.
3. Los identificadores y parámetros: ubicación, obligatoriedad y restricciones.

4. Los datos del resumen: significado, tipos, formatos y unidades.
5. Las reglas sobre importes, divisas, fechas y ausencia de información.
6. El comportamiento contractual ante errores y disponibilidad parcial de datos.
7. El mecanismo de seguridad y los permisos requeridos por operación.
8. Los componentes comunes y modelos canónicos aplicables, con sus revisiones aprobadas.
9. Los criterios de aceptación y ejemplos de solicitudes y respuestas.

Cada requisito tendrá un identificador estable para relacionarlo con el contrato y sus comprobaciones. La pertenencia a una Process API no autoriza a inventar la política de agregación ni el tratamiento de fallos parciales.

7 Organización en GitLab

GitLab alojará los contratos y permitirá revisar sus cambios mediante merge requests. La organización será por dominio y, cuando resulte útil, por capa API-Led. Cada API tendrá un proyecto de diseño independiente.

```
openfinance/
  domains/
    portfolio-management/
      experience/
        of-exp-port-investment-dashboard-v1/
      process/
        of-prc-port-investment-summary-v1/
      system/
        of-sys-port-portfolio-v1/
    account-management/
      system/
        of-sys-acct-core-banking-v1/
  platform/
    commons/
    goldenRecords/
    api-standards/
```

Los dominios y capas representan grupos o subgrupos; cada API y cada activo compartido representan proyectos. No es necesario crear proyectos para APIs que no formen parte del ejercicio.

7.1 Estructura del proyecto contractual

```
of-prc-port-investment-summary-v1/
  design/
    commons/
    goldenRecords/
    parameters/
    resources/
    rules/
    schemas/
    of-prc-port-investment-summary-v1.yaml
  validation/
    .spectral.yaml
  package.json
  package-lock.json
  redocly.yaml
  .gitmodules
  README.md
```

Esta estructura es el destino de los artefactos del taller, no una indicación de que ya existan. Se crearán únicamente los archivos y directorios necesarios para el contrato acordado.

Elemento	Responsabilidad
`design/of-prc-port-investment-summary-v1.yaml`	Documento raíz con información general, versión OpenAPI, seguridad, servidores acordados y referencias a las rutas.
`design/resources/`	Un archivo por ruta con un objeto Path Item y sus operaciones HTTP.
`design/parameters/`	Parámetros reutilizables específicos del contrato.
`design/schemas/`	Esquemas de solicitudes y respuestas propios de la API.
`design/commons/`	Submódulo Git obligatorio con componentes técnicos compartidos: errores, cabeceras y parámetros comunes.
`design/goldenRecords/`	Submódulo Git obligatorio con modelos canónicos de negocio organizados por dominio y versión.
`design/rules/`	Reglas contractuales específicas, cuando sean necesarias.
`validation/.spectral.yaml`	Configuración de Spectral y referencia a las reglas corporativas aprobadas.
`redocly.yaml`	Configuración del documento raíz y las comprobaciones de Redocly.
`package.json` y `package-lock.json`	Herramientas npm con versiones fijadas y resolución reproducible.
`.gitmodules`	Declaración de las rutas y los repositorios de los dos submódulos compartidos.
`README.md`	Propósito del contrato, dependencias e instrucciones de validación.

Los archivos de recursos se nombrarán en minúsculas y con guiones, sin llaves. Los parámetros de ruta se convertirán a palabras separadas por guiones en el nombre del archivo. Cada `$ref` será relativo al archivo que lo declara y apuntará a un objeto válido para su ubicación en OpenAPI.

No se incluirán directorios de implementación como `src`, `Application`, `Domain` o `Infrastructure`.

7.2 Componentes compartidos

`commons` contendrá estructuras técnicas reutilizables, como respuestas basadas en Problem Details, parámetros y cabeceras. `goldenRecords` contendrá modelos canónicos de negocio, no registros de datos reales ni bibliotecas de implementación.

Cada API, tanto Experience como Process o System, tendrá obligatoriamente dos submódulos Git: uno al repositorio `commons`, montado en `design/commons`, y otro al repositorio `goldenRecords`, montado en `design/goldenRecords`. No serán directorios de copias locales ni dependencias opcionales. Se referenciarán únicamente los componentes que utilice el contrato.

El proyecto declarará ambos en `.gitmodules` y fijará el commit de cada submódulo mediante las referencias registradas en el repositorio de la API. `.gitmodules` describe su configuración, pero no sustituye la fijación de esas revisiones. El equipo y Jenkins deberán inicializar y recuperar ambos submódulos antes de validar o consolidar el contrato, con permisos de lectura sobre los repositorios compartidos.

Los esquemas de entidades canónicas se referenciarán desde `goldenRecords`; las respuestas, cabeceras, parámetros y demás elementos técnicos compartidos se referenciarán desde `commons`. Los `$ref` serán relativos al archivo que los declara y apuntarán al objeto OpenAPI o esquema correspondiente. No se copiarán esas definiciones a `schemas` ni a `resources` para evitar las referencias compartidas.

7.2.1 Organización y evolución de los golden records

El repositorio `goldenRecords` organizará los modelos por dominio y, dentro de cada dominio, por versión semántica. La versión inicial será `v1.0.0`. Por ejemplo:

```
goldenRecords/
  portfolio-management/
    v1.0.0/
      portfolio.yaml
      portfolio-position.yaml
    v1.1.0/
      portfolio.yaml
      portfolio-position.yaml
  account-management/
    v1.0.0/
      financial-account.yaml
```

Esta estructura ilustra la evolución del catálogo, no archivos ya existentes. Cada directorio de versión representará un conjunto coherente y aprobado de modelos del dominio. Las referencias entre modelos también deberán respetar sus versiones explícitas, sin depender de una ruta móvil como `latest`.

Se distinguirán dos datos de trazabilidad: la versión del modelo seleccionada en la ruta y el commit fijado del submódulo, que determina qué contenido del repositorio compartido está disponible. Por ejemplo, una API podrá utilizar el modelo de `portfolio-management/v1.0.0` aunque el commit seleccionado incluya también `v1.1.0` y modelos de otros dominios.

El responsable del dominio aprobará las nuevas versiones mediante merge request. Las versiones publicadas no se modificarán en su ubicación original: las correcciones compatibles se publicarán como nuevas versiones PATCH, las ampliaciones compatibles como MINOR y los cambios incompatibles como MAJOR, tras revisar la compatibilidad contractual real. Las versiones anteriores se conservarán mientras tengan consumidores según la política de retirada acordada.

Cada API decidirá cuándo adoptar una nueva versión. El cambio actualizará mediante merge request los commits de submódulos y las rutas de los modelos que correspondan, volverá a validar las referencias y revisará el impacto sobre el contrato publicado. Un cambio en `commons` o `goldenRecords` no actualizará automáticamente los contratos consumidores.

8 Desarrollo del taller con Spec Kit

8.1 Organización de Spec Kit por API

Cada API tendrá un proyecto GitLab independiente, preparado para trabajar con Spec Kit. Cada proyecto contendrá su configuración en `.specify/`, sus especificaciones en `specs/` y el contrato OpenAPI en `design/`.

Cada proyecto dispondrá de una constitución efectiva en `.specify/memory/constitution.md`. Este archivo será el documento local de gobierno utilizado por Spec Kit, pero no se redactará ni mantendrá como una constitución independiente: se generará a partir de fuentes compartidas y de las particularidades de la API.

Preparar Spec Kit en cada repositorio no implica instalar las herramientas globales una vez por API. Lo que cada proyecto necesita es su propia configuración, su contexto y sus artefactos de trabajo.

8.2 Cuántas constituciones se deben definir

Se mantendrá una constitución corporativa para Open Finance y una constitución por dominio. Cada API tendrá únicamente un perfil de especialización propio y una constitución efectiva generada.

Documento	Cantidad	Ubicación propuesta	Mantenimiento
Constitución corporativa	Una para la organización.	Proyecto `openfinance/platform/api-standards`.	Equipo de arquitectura y gobierno de APIs.
Constitución del dominio	Una por dominio.	Proyecto `domain-governance` dentro del grupo GitLab de cada dominio.	Responsable del dominio.
Perfil de especialización	Uno por API.	`governance/api-profile.md` en el proyecto de la API.	Equipo propietario de la API.
Constitución efectiva	Una por API.	`.specify/memory/constitution.md` en el proyecto de la API.	Generada; no se edita manualmente.

Por ejemplo, con dos dominios y cuatro APIs se mantendrán tres documentos de gobierno compartido: una constitución corporativa y dos constituciones de dominio. Además, existirán cuatro perfiles de API y cuatro constituciones efectivas generadas. No habrá cuatro constituciones adicionales que deban redactarse y evolucionar de forma independiente.

Los grupos y subgrupos de GitLab organizan los proyectos y permisos, pero no transmiten automáticamente archivos o reglas entre repositorios. Pertenecer al grupo de un dominio no hace que Spec Kit cargue su constitución.

8.3 Cómo especializar los capítulos

Las constituciones compartidas asignarán identificadores estables a sus capítulos y declararán qué aspectos son obligatorios y cuáles admiten especialización. El perfil de API referenciará esos identificadores y recogerá únicamente las decisiones particulares, sin copiar el texto compartido.

Capítulo	Regla compartida	Especialización de la API
Alcance	Generar exclusivamente contratos OpenAPI.	Identificar la capacidad y la capa API-Led.
Nomenclatura	Aplicar la convención Open Finance.	Declarar el identificador de la API y su dominio.
Modelos	Reutilizar modelos canónicos aprobados.	Seleccionar los modelos utilizados y justificar representaciones específicas.
Seguridad	Cumplir los controles mínimos corporativos y del dominio.	Concretar permisos y restricciones adicionales.
Calidad	Superar las validaciones obligatorias.	Añadir comprobaciones específicas del contrato.

El dominio puede completar o reforzar las reglas corporativas, y la API puede completar o reforzar las del dominio. Ningún nivel podrá rebajar unilateralmente una obligación del nivel superior. Un conflicto bloqueará la composición hasta su resolución; no se aplicará una regla de sustitución por orden de archivos.

Las excepciones requerirán una aprobación explícita del responsable de la regla afectada, con justificación, alcance y fecha de revisión. Las operaciones, campos y comportamientos funcionales particulares se documentarán en la especificación de la API, no en la constitución.

Una API utilizará la constitución de su dominio propietario. Consumir un modelo o una capacidad de otro dominio no implica incorporar automáticamente toda la constitución de ese dominio.

8.4 Guía para las instrucciones personalizadas de Copilot

Además de la constitución efectiva, cada equipo definirá cómo orientar a Copilot en el repositorio de su API. Esta sección proporciona pautas para diseñar esas instrucciones; no contiene un archivo de instrucciones cumplimentado ni requiere generar dichos archivos durante la lectura del taller.

Se distinguirán tres responsabilidades:

Elemento	Finalidad
Constitución efectiva	Establecer los principios y obligaciones de gobierno aplicables a la API.
Instrucciones generales de Copilot	Explicar cómo trabajar en el repositorio y qué contexto consultar.
Reglas personalizadas por ámbito	Precisar las pautas aplicables a contratos, especificaciones, componentes compartidos o validaciones.

Las instrucciones no sustituirán ni copiarán la constitución. Deberán remitir a ella y explicar cómo aplicarla durante el trabajo cotidiano. No se utilizarán para aprobar excepciones de gobierno ni para introducir requisitos funcionales ausentes de la especificación.

8.4.1 Cómo estructurar las instrucciones generales

Cada proyecto utilizará `.github/copilot-instructions.md` para las pautas que deban aplicarse a todo el repositorio. Esta ubicación corresponde a la configuración de Copilot en VS Code y puede versionarse en GitLab; no implica utilizar GitHub como repositorio ni GitHub Actions como automatización.

El equipo organizará el documento con los siguientes apartados:

Apartado	Pautas que deberá definir el equipo
Propósito y alcance	Identificar la API y su capa API-Led. Limitar el trabajo a especificaciones y contratos OpenAPI, sin implementación del backend.
Contexto obligatorio	Indicar que se consulte la constitución efectiva, el perfil de API y la especificación de la evolución antes de generar artefactos.
Aclaraciones	Referenciar expresamente las reglas de aclaración del repositorio: confianza mínima del 95 %, preguntas sobre ambigüedades relevantes y prohibición de inventar requisitos.
Organización del repositorio	Explicar la función de `.specify`, `specs`, `governance` y `design`, sin reproducir toda la documentación del proyecto.
Forma de trabajar	Exigir cambios acotados, preservar el trabajo existente y señalar conflictos antes de continuar.
Archivos protegidos o generados	Identificar la constitución generada, las dependencias compartidas y el consolidado como elementos que no deben editarse manualmente desde la API.
Validación y entrega	Remitir a los comandos aprobados y documentados del proyecto, exigir evidencias y separar la generación del contrato de su publicación mediante Jenkins.

El archivo será breve, operativo y específico del proyecto. No incluirá grandes catálogos de reglas OpenAPI, copias completas de la constitución ni comandos que el equipo no haya comprobado. Las instrucciones que se creen se redactarán en inglés, conforme a la política del repositorio; esta guía se mantiene en español.

8.4.2 Qué reglas personalizadas añadir

Las pautas que solo correspondan a ciertos archivos se organizarán en `.github/instructions/`, con una responsabilidad por archivo. Cada regla tendrá una descripción de cuándo utilizarla y un ámbito `applyTo` ajustado a las rutas pertinentes.

Regla propuesta	Ámbito orientativo	Pautas que conviene recoger
Diseño contractual OpenAPI	Archivos YAML del contrato bajo `design`.	Documento raíz único, Path Items en `resources`, separación de parámetros y esquemas, referencias relativas, nomenclatura y ejemplos coherentes.
Especificaciones y aclaraciones	Documentos bajo `specs`.	Aplicación de las 7 C, requisitos identificables, criterios de aceptación observables y conservación de las decisiones aclaradas.
Componentes compartidos	Contenido bajo `design/commons` y `design/goldenRecords`.	Uso de revisiones aprobadas, ausencia de copias locales divergentes y tramitación de cambios en el repositorio propietario.
Validación y consolidación	Configuraciones de validación y herramientas.	Versiones fijadas, validación con Spectral y Redocly, conservación de informes y prohibición de desactivar controles para ocultar errores.
Documentación	Documentos del proyecto, según su función.	Redacción clara, idioma acordado, enlaces verificables y distinción entre diseño, publicación e implementación.

El equipo concretará los patrones al preparar los archivos; no aplicará todas las reglas a todos los documentos. Las pautas de diseño contractual deberán respetar la protección de los componentes compartidos cuando ambos ámbitos coincidan. No se asumirá que una instrucción específica anula automáticamente una general.

8.4.3 Reutilización y comprobación

Se mantendrá una base versionada de pautas en `api-standards`. Cada API adoptará una revisión aprobada y completará únicamente su contexto local, mediante merge request. Esta reutilización no será automática por pertenecer a un grupo de GitLab y no deberá crear otra jerarquía de constituciones independientes.

Antes de comenzar a trabajar, el equipo comprobará en VS Code que las instrucciones están habilitadas y que las pertinentes se incorporan al contexto de Copilot. La mera referencia a un documento remoto no garantiza su lectura: los documentos necesarios deberán estar disponibles y consultarse expresamente.

Las instrucciones orientan al asistente, pero no garantizan por sí solas el cumplimiento. Las reglas verificables deberán seguir comprobándose mediante Spectral, Redocly y Jenkins. Cualquier contradicción entre instrucciones, constitución y especificación deberá resolverse antes de generar el contrato.

8.5 Paso 1. Preparar el gobierno y generar la constitución efectiva

El equipo realizará las siguientes actividades antes de redactar la especificación:

1. Definir o seleccionar la constitución corporativa: alcance exclusivamente contractual, nomenclatura, estructura de diseño, seguridad mínima, errores y validaciones obligatorias.
2. Definir o seleccionar la constitución del dominio: terminología, modelos canónicos, restricciones y criterios de revisión específicos. Esta actividad se realiza una vez por dominio, no una vez por API.
3. Preparar el proyecto de la API con Spec Kit y redactar su perfil de especialización, referenciando los capítulos compartidos aplicables.
4. Registrar en el proyecto las revisiones Git exactas de las fuentes corporativas y del dominio. No se utilizarán ramas móviles como única referencia de gobierno.

5. Componer las reglas compartidas y el perfil local, comprobar conflictos y generar `.specify/memory/constitution.md` con los capítulos completos aplicables.
6. Registrar en el documento generado la procedencia de las fuentes y sus revisiones, y versionarlo junto con el perfil y las referencias fijadas en el repositorio de la API.
7. Revisar la constitución efectiva antes de utilizarla como contexto para los siguientes pasos de Spec Kit.
8. Definir los apartados de las instrucciones generales y seleccionar las reglas personalizadas necesarias según la guía anterior. Cuando se preparen sus archivos, comprobar su coherencia con la constitución y su aplicación en VS Code.

Para `of-prc-port-investment-summary-v1`, las fuentes serán la constitución corporativa de Open Finance, la del dominio `portfolio-management` y el perfil particular de esta Process API.

La composición descrita es una automatización propuesta para este modelo de gobierno; no debe asumirse como una función de herencia automática de Spec Kit. Deberá estar disponible tanto en el entorno local como en Jenkins antes de adoptar este flujo. Este taller documenta el diseño del mecanismo, no implementa dicha automatización.

No bastará con generar la constitución en Jenkins: deberá estar disponible localmente antes de especificar, planificar o generar el contrato con Spec Kit. La constitución efectiva indicará expresamente que no se generará código de implementación de la API y conservará la regla de aclaraciones del repositorio.

8.6 Actualización de las reglas compartidas

Los cambios corporativos o de dominio se revisarán mediante merge requests en sus repositorios de origen. Cada API adoptará una nueva revisión mediante un merge request propio, regenerará su constitución efectiva y revisará el impacto sobre sus especificaciones y su contrato. Una modificación compartida no actualizará silenciosamente los proyectos consumidores.

Los cambios de gobierno se realizarán en la fuente correspondiente o en el perfil local, nunca editando directamente la constitución generada. El flujo de actualización de constitución de Spec Kit deberá respetar esta separación para no sobrescribir el documento efectivo de forma independiente.

Jenkins recuperará las fuentes fijadas, reproducirá la composición y comprobará que coincide con la constitución versionada. Esta comprobación detectará alteraciones manuales y fuentes inconsistentes; no sustituirá la revisión de cumplimiento de las reglas de negocio y gobierno.

8.7 Paso 2. Redactar la especificación

Utilizar el flujo de especificación de Spec Kit para describir las APIs que deben diseñarse. Revisar el texto con las 7 C y registrar criterios de aceptación observables. El resultado debe dejar explícitos los endpoints y las entidades canónicas o golden records utilizados por cada API, antes de generar sus contratos.

8.7.1 Identificar el dominio y las APIs necesarias

El equipo identificará el dominio propietario, su código de nomenclatura, el responsable y el grupo padre de GitLab. Si se necesitan APIs de diferentes dominios, se repetirá la identificación para cada una y se respetará su propiedad; no se ubicarán todas bajo el dominio de la API de experiencia por conveniencia.

Se elaborará un inventario con el identificador de cada API, su capa, propósito, consumidores, dependencias y proyecto GitLab previsto. La Process API deberá justificarse mediante una necesidad de orquestación. Si no existe esa necesidad, se documentará la relación directa entre Experience y System y no se planificará una API de proceso.

8.7.2 Especificar los endpoints

Para cada API, el equipo completará un catálogo de operaciones que incluya:

- Identificador del requisito y de la operación, método HTTP, ruta exacta y propósito.

- Parámetros de ruta, consulta y cabecera, con tipos, formatos, obligatoriedad y restricciones.
- Cuerpo de solicitud, cuando corresponda, y esquemas de respuesta por código HTTP.
- Reglas de negocio observables, seguridad, permisos, errores y casos límite relevantes.
- Ejemplos coherentes y criterios de aceptación que permitan comprobar el resultado.
- Dependencias contractuales con otras APIs y comportamiento ante información parcial, cuando proceda.

No será suficiente indicar que una API «gestiona carteras» o «devuelve un resumen». Cada endpoint deberá quedar definido con precisión suficiente para construir su Path Item y sus operaciones sin inventar decisiones.

8.7.3 Identificar los golden records

Para cada entidad canónica se registrarán su nombre, significado, dominio propietario, revisión aprobada, ubicación del esquema y operaciones que la utilizan. Se explicarán sus atributos, tipos, formatos, obligatoriedad, relaciones y restricciones aplicables al contrato.

Se distinguirán tres situaciones:

Situación	Actuación
El modelo canónico existe y cubre la necesidad.	Referenciar la revisión aprobada desde `design/goldenRecords`, sin copiar su definición a esquemas locales.
El modelo existe, pero requiere una representación específica de la API.	Justificar la transformación o el esquema propio, manteniendo la trazabilidad semántica y sin modificar silenciosamente el modelo compartido.
La entidad canónica no existe o requiere cambios.	Proponer su definición o evolución al responsable del repositorio compartido y obtener aprobación antes de generar un contrato que dependa de ella.

No se confundirá una entidad de negocio con los elementos técnicos de `commons`, como errores, cabeceras o paginación. Cada API referenciará únicamente las entidades canónicas necesarias para sus operaciones.

Como salida del paso se dispondrá de una matriz de trazabilidad: API, dominio, capa, endpoint, requisito, golden records, componentes comunes y criterios de aceptación. Esta matriz será la entrada del agente de generación contractual.

8.8 Paso 3. Resolver las ambigüedades

Utilizar el flujo de aclaraciones para resolver decisiones pendientes. No comenzar la generación contractual mientras falten datos necesarios sobre operaciones, modelos, seguridad o errores. El equipo deberá aprobar el inventario de APIs, el catálogo de endpoints y los golden records aplicables antes de pasar a la generación.

8.9 Paso 4. Preparar GitLab y planificar el diseño contractual

Definir la distribución de rutas, parámetros y esquemas; identificar las referencias compartidas; y acordar las comprobaciones con Spectral y Redocly. El plan debe producir artefactos de diseño y validación, no una arquitectura de implementación.

8.9.1 Comprobar el dominio y preparar los proyectos

El agente, con las herramientas y permisos disponibles, seguirá este procedimiento:

1. Confirmar la instancia de GitLab, el grupo padre `openfinance/domains`, el dominio y las rutas de los proyectos que figuran en la especificación. No inventará direcciones ni solicitará credenciales mediante el chat.

2. Comprobar si existe el subgrupo del dominio bajo ese grupo padre. Distinguirá entre inexistencia, falta de permisos y fallo de conexión; no interpretará un acceso denegado como autorización para crear otro dominio.
3. Si el dominio no existe, preguntar expresamente si debe crearlo, indicando nombre y ruta. Solo tras la confirmación creará el subgrupo. Si se rechaza, detendrá las operaciones dependientes y conservará el trabajo de especificación.
4. Localizar el proyecto `domain-governance` del dominio y clonar su repositorio cuando exista. Si falta, solicitar aprobación para crearlo y preparar su gobierno y sus recursos desde las bases corporativas aprobadas.
5. Revisar los proyectos de API existentes. Reutilizará y clonará los que correspondan al inventario; no duplicará proyectos ni sobrescribirá directorios locales con cambios.
6. Presentar los subgrupos de capa y los proyectos de API que falten. Con la autorización correspondiente, crear únicamente `experience`, `system` y, si está justificado, `process`, bajo el dominio propietario de cada API.
7. Crear un proyecto GitLab por API dentro de la capa correspondiente, siguiendo la nomenclatura Open Finance. Cada vez que se cree un proyecto de API, clonar su repositorio y añadir, como parte obligatoria de su preparación inicial, los dos submódulos Git: `design/goldenRecords`, para las entidades canónicas, y `design/commons`, para los componentes comunes. Preparar también Spec Kit, la constitución efectiva, las instrucciones y la estructura `design` antes de generar el contrato. Esta regla se aplicará a todas las APIs de Experience, Process y System.
8. Registrar los dos submódulos en `.gitmodules` y sus referencias a commits aprobados en el repositorio de la API. Inicializarlos y comprobar que contienen las versiones por dominio seleccionadas y que las referencias serán resolubles tanto localmente como en Jenkins. Si se reutiliza un proyecto existente, verificar ambos submódulos y añadir únicamente los que falten, sin duplicar los ya configurados.
9. Registrar las rutas e identificadores de los recursos creados o reutilizados. Una nueva ejecución deberá reconocerlos y evitar duplicaciones.

Un subgrupo de GitLab es un contenedor organizativo, no un repositorio que pueda clonarse. Se clonarán los proyectos que contiene: el repositorio de gobierno del dominio y cada repositorio de API requerido.

Si no se necesita una Process API, no se creará el subgrupo `process` para este trabajo. Si ese subgrupo ya existe por otras capacidades, no se eliminará ni modificará por ese motivo.

Si el dominio es nuevo, la preparación inicial del paso 1 será provisional y local. Tras autorizar su creación, se establecerán sus fuentes de gobierno y se completará la constitución efectiva de cada proyecto antes de generar contratos. Si faltan herramientas de GitLab o permisos, el agente informará del bloqueo y de la operación pendiente; no afirmará haberla ejecutado.

8.10 Paso 5. Agente Implementar contrato y skills necesarias

En este taller, «implementar el contrato» significa generar los documentos OpenAPI y sus componentes, no implementar los servicios de backend. El equipo preparará un agente con esa responsabilidad y skills reutilizables con procedimientos acotados. Esta sección es una guía para diseñarlos; no contiene sus archivos cumplimentados ni presupone que ya estén disponibles.

8.10.1 Dónde mantener el agente y las skills

El proyecto `domain-governance` será la fuente versionada de estos recursos para todas las APIs del dominio:

```
domain-governance/
  .github/
    agents/
      implement-api-contract.agent.md
    skills/
      specify-api-contract/
        SKILL.md
      prepare-gitlab-api-projects/
        SKILL.md
      resolve-canonical-models/
        SKILL.md
      generate-modular-openapi/
        SKILL.md
      validate-bundle-openapi/
        SKILL.md
```

Los nombres son propuestas para los recursos que el equipo debe preparar. El agente podrá mostrarse como `Implement API Contract`, identificado en el guion como «Agente Implementar contrato». Sus instrucciones, descripciones y skills se redactarán en inglés y referenciarán explícitamente la regla de aclaraciones y el alcance exclusivamente contractual.

Clonar el repositorio de gobierno no garantiza que VS Code descubra sus personalizaciones en el contexto de otra API. El equipo establecerá un mecanismo de distribución versionada que sitúe los archivos aprobados en las ubicaciones reconocidas del proyecto consumidor y registre su procedencia. Comprobará su disponibilidad en VS Code antes de utilizarlos. No se mantendrán copias divergentes editadas manualmente en cada API.

8.10.2 Responsabilidades del agente

> **GUARDRAILS DEL AGENTE**
>
> El agente leerá la constitución efectiva, el perfil de API, las instrucciones aplicables y la especificación aprobada. Recibirá el inventario de APIs, endpoints y golden records, comprobará que no quedan decisiones relevantes pendientes y coordinará las skills necesarias.
>
> Dispondrá únicamente de las herramientas necesarias para leer el contexto, consultar GitLab, editar contratos y ejecutar validaciones. La creación de recursos remotos requerirá la autorización prevista en el paso 4. No publicará contratos, no modificará gobierno compartido ni generará código de backend sin una solicitud independiente y explícita.

8.10.3 Skills que debe preparar el equipo

Skill propuesta	Cuándo se utiliza	Resultado y control
`specify-api-contract`	Durante los pasos 2 y 3.	Inventario de APIs, endpoints y criterios revisados con las 7 C; preguntas sobre decisiones pendientes. Complementa el flujo de especificación de Spec Kit, no lo sustituye.
`prepare-gitlab-api-projects`	Al preparar el dominio, las capas y los proyectos.	Comprobación de existencia, propuestas de creación, confirmaciones, clonación de proyectos y registro de recursos sin duplicaciones.
`resolve-canonical-models`	Antes de generar esquemas y referencias.	Comprobación de los dos submódulos obligatorios, selección de golden records por dominio y versión, componentes comunes aprobados, commits fijados y detección de necesidades de evolución.

Skill propuesta	Cuándo se utiliza	Resultado y control
`generate-modular-openapi`	Cuando endpoints y modelos están aprobados.	Documento raíz, Path Items, parámetros y esquemas propios por API, siguiendo la estructura `design`.
`validate-bundle-openapi`	Después de cada parte y al completar la API.	Validaciones pertinentes con Spectral y Redocly, resolución de referencias, consolidado final e informes trazables.

Cada skill describirá sus entradas, requisitos previos, pasos, herramientas, condiciones de parada y evidencias de salida. Las herramientas de GitLab deberán verificarse en el entorno real; nombrar una skill no le concede acceso ni permisos. Las reglas de gobierno permanecerán en sus fuentes, sin duplicarlas en cada skill.

8.10.4 Generar cada contrato por partes

Con el plan y las tareas de Spec Kit aprobados, el agente procederá por API:

1. Confirmar su dominio, capa, identificador y proyecto de destino. Si el proyecto de API es nuevo, añadir los dos submódulos Git obligatorios durante su preparación inicial: `design/goldenRecords`, para referenciar las entidades canónicas, y `design/commons`, para referenciar los componentes comunes. No se dejará su incorporación para una fase posterior. Si el proyecto ya existe, comprobar su configuración y añadir únicamente los submódulos que falten. Antes de generar recursos, ambos deberán estar registrados en `.gitmodules`, inicializados y fijados a commits aprobados en el repositorio de la API. Preparar un único documento raíz bajo `design`.
2. Seleccionar explícitamente el dominio y la versión de cada entidad canónica, empezando por `v1.0.0` cuando sea la versión aprobada aplicable. Construir los `$ref` a los archivos del submódulo `goldenRecords` y a las partes comunes del submódulo `commons`, sin duplicar sus definiciones. Crear bajo `schemas` únicamente representaciones propias justificadas por la especificación y bajo `parameters` los parámetros reutilizables específicos de la API.
3. Generar un archivo en `resources` por ruta acordada, con sus operaciones HTTP, entradas, respuestas, seguridad y referencias. No reunir varias APIs independientes en un mismo contrato.
4. Incorporar cada Path Item al documento raíz y comprobar la parte generada antes de continuar. Cualquier validación sobre un contrato todavía incompleto deberá identificarse como parcial, sin afirmar que el contrato final ya está aprobado.
5. Repetir el procedimiento para los recursos restantes y comprobar la cobertura de todos los endpoints y entidades de la matriz de trazabilidad.
6. Ejecutar la validación completa, generar el consolidado de esa API y volver a validarlo. Preparar las evidencias para la revisión y la pipeline posterior de publicación.
7. Repetir el flujo en los proyectos de Experience, Process y System incluidos en el inventario, sin crear capas adicionales. Planificar las dependencias contractuales para mantener consistentes las representaciones entre APIs.

Los archivos de cada contrato residirán en su proyecto, dentro del subgrupo de capa; no directamente en el subgrupo de GitLab. El resultado será un contrato modular y un consolidado por API, con los golden records y endpoints acordados. Todos los proyectos de API conservarán sus dos submódulos, aunque utilicen distintas entidades, dominios o versiones; no se creará un submódulo adicional por cada dominio o versión de golden records.

La generación no añadirá operaciones, campos, respuestas ni dependencias que no estén justificados por la especificación.

8.11 Paso 6. Validar y consolidar

Ejecutar las comprobaciones contractuales, resolver los errores y generar el OpenAPI consolidado. Revisar tanto la validez técnica como la cobertura de requisitos: un linter no demuestra por sí solo que el contrato represente correctamente el negocio.

8.12 Paso 7. Revisar y aprobar

Presentar la especificación, el contrato modular, la trazabilidad y los informes en un merge request. El contrato se aprobará cuando cumpla los criterios de aceptación y no presente errores bloqueantes.

9 Validación y consolidación con Jenkins

Jenkins será el orquestador. Las definiciones de los pipelines se mantendrán en el repositorio de automatización acordado; no se generará configuración de GitLab CI/CD.

Etapa	Herramienta o responsable	Resultado esperado
1. Recuperar fuentes	Jenkins y Git	Revisión del contrato y sus dos submódulos obligatorios inicializados en los commits fijados, con las versiones de modelos referenciadas disponibles.
2. Preparar herramientas	npm	Dependencias instaladas de forma reproducible desde el archivo de bloqueo `package-lock.json`.
3. Validar gobierno	Stoplight Spectral	Informe de cumplimiento de las reglas acordadas.
4. Validar OpenAPI	Redocly CLI	Documento modular válido y referencias resolubles.
5. Consolidar	Redocly CLI	Un único contrato generado siguiendo las referencias del documento raíz.
6. Validar el consolidado	Spectral y Redocly CLI	Artefacto final sin errores bloqueantes.
7. Conservar evidencias	Jenkins	Informes y artefacto identificados por revisión y versión, con su huella de integridad, por ejemplo `sha256`.

Para el caso práctico, el archivo de entrada será `design/of-prc-port-investment-summary-v1.yaml` y el artefacto generado será `dist/of-prc-port-investment-summary-v1.yaml`.

REGLA DE ORO DE LA PIPELINE

> El documento modular es la fuente de verdad; el consolidado se conserva como artefacto generado, sin editarlo manualmente.
>
> - La consolidación no combinará contratos de APIs independientes.
> - Las versiones de OpenAPI, Node.js, Spectral y Redocly se fijarán antes del ejercicio y se comprobará su compatibilidad. No se utilizarán versiones flotantes ni direcciones de servidores ficticias para completar información pendiente.

- Los errores de sintaxis, las referencias inexistentes, los identificadores de operación duplicados y los incumplimientos de reglas obligatorias bloquearán la aprobación. La política de advertencias se acordará previamente.
- Los informes disponibles se conservarán también cuando falle una etapa.

Para evoluciones posteriores se incorporará una comparación de compatibilidad respecto de la versión publicada. El linting no detecta por sí solo todos los cambios incompatibles.

9.1 Uso del contrato en la pipeline de publicación

El contrato consolidado y validado será el artefacto de entrada de una pipeline de Jenkins dedicada a importar o actualizar la definición de la API en el gestor seleccionado. Los destinos contemplados incluyen Amazon API Gateway en AWS, WSO2 API Manager y Azure API Management. La elección del producto no modificará la organización de los contratos fuente en GitLab.

Se distinguirán dos flujos con responsabilidades separadas:

Flujo	Entrada	Salida
Validación y consolidación	Contrato modular y dependencias fijadas.	Contrato consolidado y validado, con informes, versión, revisión Git y huella de integridad.
Publicación	Artefacto aprobado y configuración del destino.	Definición importada o actualizada en el gestor de APIs, con evidencias de la operación.

La pipeline de publicación deberá:

1. Recuperar el artefacto aprobado por su identificador de versión y comprobar su integridad mediante la huella registrada (por ejemplo, `sha256`) y las evidencias de validación. No reconstruirá el contrato desde una rama que pueda haber cambiado.
2. Comprobar la compatibilidad de la versión y las características OpenAPI utilizadas con el producto y la versión de destino. Superar Spectral y Redocly no garantiza por sí solo que un gestor acepte el contrato.
3. Incorporar, cuando proceda, la configuración específica del entorno: integraciones con backends, políticas, permisos y otros ajustes del gestor. Esta configuración se mantendrá separada del contrato fuente y no contendrá secretos versionados.
4. Si el destino requiere transformar el contrato o añadir extensiones específicas (por ejemplo, `x-amazon-apigateway-*`), generar un artefacto derivado trazable y volver a validarlo antes de su importación. No se modificará manualmente el consolidado aprobado.
5. Solicitar las aprobaciones exigidas para el entorno e importar o actualizar la definición con una identidad de permisos mínimos.
6. Comprobar el resultado y registrar el gestor, el entorno, la versión y la huella del artefacto utilizado, sin alterar el repositorio fuente en GitLab. Cualquier activación o exposición al tráfico se realizará mediante los controles previstos para ese destino.

Importar un contrato no implementa la API ni garantiza que exista un backend operativo. La disponibilidad del servicio dependerá de las integraciones y de los servicios configurados en una iniciativa separada. Este taller define la entrega entre ambos flujos, pero no implementa ni ejecuta la pipeline de publicación.

10 Criterios de aceptación

Para dar por completado el taller, el entregable debe cumplir todos los criterios siguientes:

- La especificación cumple las 7 C y no contiene decisiones contractuales relevantes pendientes; se alcanza al menos un 95 % de confianza.
- Cada API tiene identificado su dominio propietario, su catálogo de endpoints y los golden records aprobados que utiliza, con trazabilidad entre ellos.
- Los subgrupos y proyectos GitLab se crean únicamente cuando faltan y existe autorización; no se crea una capa Process sin necesidad de orquestación.
- El agente y las skills del dominio tienen una revisión identificable y están disponibles en el contexto de trabajo antes de generar contratos.
- La capa API-Led y la nomenclatura de Open Finance son coherentes en todos los artefactos.
- El contrato se encuentra bajo `design` y dispone de un único documento raíz.
- Cada ruta referencia un archivo de recursos con un Path Item válido y sus operaciones HTTP.
- Todas las operaciones tienen identificadores únicos, descripciones, entradas y respuestas acordes con los requisitos.
- Los esquemas, restricciones, formatos, campos obligatorios y ejemplos son consistentes.
- La seguridad y los errores están definidos conforme a las decisiones aprobadas.
- Las referencias compartidas se resuelven desde revisiones identificables y reproducibles.
- Cada API declara e inicializa los dos submódulos obligatorios, `design/commons` y `design/goldenRecords`, con commits fijados y sin duplicar sus definiciones.
- Los golden records utilizados se identifican por dominio y versión explícita; su evolución no modifica las versiones publicadas ni actualiza automáticamente las APIs consumidoras.
- Spectral y Redocly completan las validaciones sin errores bloqueantes.
- Jenkins recupera las fuentes fijadas, instala dependencias desde el lockfile, ejecuta las validaciones y conserva informes y artefactos asociados a la revisión, versión y huella de integridad; una etapa fallida conserva las evidencias disponibles.
- El contrato consolidado se genera y vuelve a validarse correctamente.
- El artefacto consolidado queda identificado por versión, revisión Git y huella de integridad, con sus evidencias de validación, para que la pipeline de publicación pueda recuperarlo sin reconstruirlo.
- Se documentan el gestor de APIs de destino y las comprobaciones de compatibilidad requeridas antes de importar el contrato.
- Existe trazabilidad entre requisitos, operaciones y evidencias de revisión.
- No se ha generado código de implementación de la API ni desplegado ningún servicio.

11 Resultado esperado

CONCLUSIÓN Y ENTREGABLE FINAL

Al finalizar, el equipo dispondrá de una especificación funcional revisada y de su contrato OpenAPI modular y consolidado, con evidencias de validación. El consolidado aprobado se utilizará como entrada de la pipeline de publicación en Amazon API Gateway, WSO2 API Manager o Azure API Management, según el destino seleccionado y sus requisitos de compatibilidad. La implementación del backend y la ejecución de la publicación seguirán siendo actividades independientes del taller.